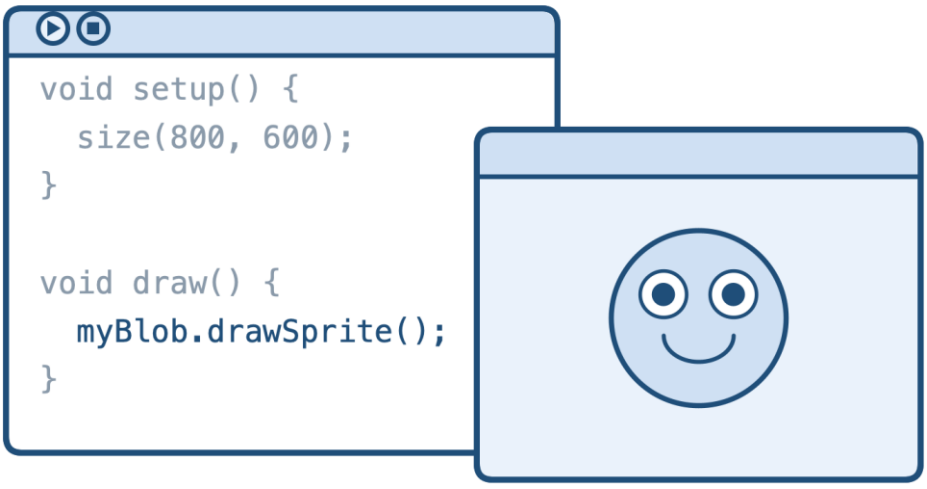
Prologue - How This Book Works

Your character, your world, your rules.



In this book you build a video game: an actual game, with a character you draw yourself, that you can hand to somebody else and watch them play.

You start on the first day by putting something on the screen. In Chapter 2 you draw your character. After that, every chapter gives that character something new. It moves, then you control it, then it gets a world to live in, then things to collect and things to avoid. By Chapter 7 the game is yours: your character, your world, your rules.

You can start this book having never seen a line of code. It begins at the very beginning.

Objects first

The first thing you make in this book is an **object**: a thing in your program that has its own information and knows how to do things. Plenty of programming courses spend weeks on other things before they get to objects. This one starts with an object on the first day, and every chapter after that is about objects: using them, then making your own, then making them do more.

You'll use objects before you know everything inside them. That's on purpose. A Blob already knows how to draw itself, how to move, and how to tell when it's touching something. You'll ask it to do those things, and see them work, a good while before you'd know how to build them yourself. Curiosity about how it works is the bridge to building your own.

When I was in college, I had a textbook called *Objects First with Java*. I didn't realize it at the time, but that book settled my belief that objects are the right place to start, and this book starts with an object for the same reason.

Where this came from

The objects in this book come from a library called **SpriteGame**. I wrote it for Coding for Embedded Systems, the sixth-grade class I teach at Winsor. The class runs for two quarters. The first quarter is Processing, and that's this book. The second is Arduino, on the Circuit Playground board.

SpriteGame comes from the same belief about objects first that I teach my high school class with. The version that went into its repository in 2021 was already about its third, and in the spring of 2026 it became a library you can install into Processing.

The first quarter is Processing because Processing draws. You write a few lines, press Run, and something shows up on your screen, fast. That's a good way to start.

How the chapters got written

The chapters came after the lessons. Each lesson has a plan, a page on Canvas, and the code you start from. Each chapter got written from those three things.

So the code in every chapter is real. It's copied from what the lesson gives you, or it's code the chapter asks you to write yourself. Every picture of a sketch window in this book is a real one: the code on the page was run, and the picture is what it drew. When a chapter shows an error message, that's the message Processing printed, word for word.

I wrote this book with a writing assistant: Claude, an AI model made by Anthropic. Claude helped me turn each lesson, its code, and its Canvas page into the chapter you're reading. This book could not exist without that help.

This is the first year the course runs this way, and the book is being written while it runs. Like the course, it'll change the next time.

How a chapter runs

Most chapters take one class meeting. Chapter 6 takes two, and Chapter 7 takes three. All of the building happens in class.

Your character

From Chapter 2 on, there's one tab in your sketch that belongs entirely to you: `MyCharacter`. Every chapter does its new idea once with an example, so everybody has the same working code and the same words for it. Then it hands you the idea to try on your character. That's where you find out whether you can do it alone.

Run it after every change

Each chapter's building is cut into small Steps, and your sketch runs at the end of every one. Run it after every change. That's how you find out what you broke, while it's still one line.

The chapters come in two units. Unit 1, **Your Character**, is Chapters 1 to 4: you make a character, make it move, and take the controls. Unit 2, **Your Game**, is Chapters 5 to 7: a world, things to collect and avoid, and a game that ends.

Unit	Chapter	Meetings
1	1 - Hello, Blob!	1
1	2 - Draw Your Character	1
1	3 - Bring It to Life	1
1	4 - You're in Control	1
2	5 - Build Your World	1
2	6 - Collisions: If This, Then That	2
2	7 - Build Your Game	3

Four bonus chapters come at the back of the book: images, sounds, animation, and lists. Each one says when it's best taken. After them come three appendices. **Appendix 1** is for you: how to save your sketch, find it in Finder, zip it, and hand it in on Canvas, with fixes for when Finder looks wrong. Appendices 2 and 3 are for teachers. Appendix 2 shows how the library works on the inside. Appendix 3 prints everything Sparky is told.

Sparky

When you get stuck, there's **Sparky**, an AI helper on Flint. Every lesson page and Assignment on Canvas has a button that opens it. Each lesson has its own Sparky, which knows that lesson's code and the words you've learned so far.

Sparky has three jobs.

- **Syntax.** The exact spelling, a missing semicolon, what a red error message means. Little things like these can eat a whole class, and none of them is what the chapter is about.
- **Ideas.** When you're stuck on what to make, Sparky can give you two or three to pick from.
- **The math.** If you want a hat sitting right on top of your character's head, Sparky can work out the numbers. Then you get back to building.

Some things are yours, and Sparky leaves them to you. You decide what your character looks like, what your world is, and what your game is about. You type the code in yourself. And when a semicolon goes missing, Sparky helps you find it on your own: read the red message, go to the line Processing highlights, and check the end of the line above it. Chapter 1 shows you how.

Sparky usually hands you code with a few blanks in it. You fill them in, type the code, and run it. The first students to take this course said that was the help that worked best, so it's how every Sparky works now. If you're still stuck, ask again, and Sparky shows you the whole thing.

Asking Sparky

Sparky works best when you tell it which chapter you're on and paste in your code. Ask for one step at a time. For a shape, describe it in plain words: where it starts, where it ends, and which way it curves. Every chapter has an **Ask Sparky** box with good questions to start with.

Before you paste in a big change, save a copy with **File → Save As** and a number on the end of the name. Then your old version is still there if something breaks.

Sparky can be wrong. If it gets something wrong twice, or says it isn't sure, take it to your teacher.

One more Sparky is on the **Saving and Handing In Your Work** page on Canvas. It helps you find your sketch, zip it, and hand it in. It doesn't do code.

Nothing about Sparky is secret. Appendix 3, at the very back of this book, prints everything it's told.

How to read a chapter

The shape

Every chapter has the same parts, in the same order.

- **An opening** that says where the last chapter left you, and what's still missing. A table names the chapter's two or three moves.
- **Part One** reads the code you start from. Its sections are numbered by chapter, 1.1, 1.2, 1.3, and each one takes apart one piece of it.

- **Part Two and after** build. The building comes in numbered Steps, and each Step changes one or two things.
- **The end of the chapter** gathers it up: a summary of what you built, a table of new code and where you met it, the vocabulary, a few questions to check your understanding, and a look at what comes next.

The marks

Build it marks code to type into your sketch. Type it by hand when you can. Your fingers learn the code faster than your eyes do.

Your turn marks code you write yourself. The chapter says what it has to do, and leaves the how to you.

Optional marks a **Your turn** that nothing later depends on. Skip it if you're short on time, and the steps after it still work. Do it if you're curious.

The rest come in boxes, and each kind has its own color.

Aside: what an Aside is

An Aside explains a new piece of code the first time it shows up in the book. If you already know it, skim the box and keep going.

Check. A Check marks a place to stop, run your sketch, and make sure it does what the chapter says. It tells you what you should see, and most Checks come with a picture of the window. If yours is different, fix that before you go on, because the next step builds on this one.

Watch out. A Watch out box is the mistake nearly everybody makes at that spot, and what it looks like when you make it.

Argue it. A question with more than one good answer. Pick one, and be ready to say why.

Today's example. From Chapter 2 on, every chapter starts with this box. It names the lesson's example, its finished code, which you open from Processing's **File → Examples** menu. Run it to see where the chapter is headed, and keep it open to compare against your own sketch when it does something strange.

Ask Sparky. Every chapter has this box near the start. It gives a few good questions for that lesson's Sparky, and says what to paste in with them.